

CONEXA: Online Conformance and Exhibition-Driven Model-Based Testing

No Author Given

No Institute Given

Abstract. Model-based testing (MBT) commonly assesses whether a system conforms to its specification. However, conformance alone does not guarantee that all required behaviors have manifested. Test purposes address this limitation by defining observation objectives that a system should exhibit during testing. This paper presents **Conexa**: an online Conformance-Exhibition Algorithm that combines conformance testing with exhibition-driven testing. **Conexa** uses a specification model and a test purpose to guide test generation towards the exhibition of requirement-related behavior. **Conexa** explores the specification model on-the-fly and therefore avoids exhaustive state-space exploration. To improve performance, it incorporates heuristics for goal-directed exploration, transition prioritization, and trace diversification. To assess its applicability to real-world settings, we implement **Conexa** in a commercial MBT tool. Experimental results show that, given informative test purposes, **Conexa** exhibits targeted behaviors more frequently and in fewer steps than random and transition-coverage-based strategies.

Keywords: Model-based testing · test purposes · online testing · symbolic transition systems · requirements coverage

1 Introduction

Testing and verification of software is as important as it is time-consuming. Model-based testing (MBT) is an automation technique in which test cases are generated from a formal model of the system under test [3]. These test cases check *conformance*: whether the system behaves according to its specification. Correctness, however, also requires *exhibition*: the ability to observe all behavior implied by the requirements [31]. Test purposes capture such behavioral objectives and relate test cases to the requirements they are intended to validate [31].

Several approaches have been proposed for incorporating test purposes into MBT. As in traditional MBT, test cases are generated by traversing a formal model that captures the intended system behavior. The key difference is that test purposes guide this traversal, steering test generation toward traces that satisfy specific behavioral objectives. Existing research has predominantly taken a formal approach, focusing on the definition and analysis of sound methodologies.

Despite their potential, test purposes are *not* standard practice in MBT, where structural coverage-based and random strategies dominate [18]. To investigate their applicability in industrial settings, we implement test purposes as a

test-generation strategy in a commercial MBT tool and evaluate them empirically rather than formally. Because models of real-world systems often suffer from state-space explosion due to factors such as large data domains, concurrency, and composition of subsystems, we employ an online approach that explores the model on-the-fly rather than offline.

2 Background

2.1 Conformance and exhibition

Conformance testing determines, through test execution, whether an implementation behaves according to its specification. In formal testing theory [26], test execution acts as a decision or semi-decision procedure for a conformance relation: an implementation is considered conformant if it passes the relevant tests. An *observation objective* on the other hand, is a behavior, i.e., the sequence of events, that a tester wishes to observe during testing [31]. An implementation *exhibits* an observation objective if it is capable of producing the behavior described by that objective.

Conformance captures safety properties (something bad never happens), whereas exhibition captures liveness properties (something good can eventually happen). Together they provide a more complete assessment of system correctness. Consider for instance a coffee machine that should dispense a specific drink for each button pressed, with the following trace:

?button_coffee → !coffee_dispensed → !button_tea → ?tea_dispensed
→ !button_chocolate → !chocolate_dispensed

This trace conforms to the specification, yet does not exhibit all required behavior since no cappuccino is dispensed. Table 1 combines conformance and exhibition verdicts. (Pass, Hit) gives the strongest confidence in correctness [31].

Table 1: Test case verdicts

	Conformant	Non-conformant
Observation objective exhibited	Pass, Hit	Fail, Hit
Observation objective not exhibited	Pass, Miss	Fail, Miss

2.2 Model-based testing, the STS, and ioco

In MBT, test cases are generated automatically from a formal behavioral model of the system under test (SUT) [3]. The model also functions as a test oracle by defining a conformance relation between the SUT and its specification [24]. Test cases are automatically assigned a verdict, i.e., *pass* or *fail*, based on this conformance relation. We consider the SUT as a black box: we interact with it and observe its behavior without any knowledge about its internals. Interaction is done via *stimuli* (inputs), and the observed outputs are called *responses*.

Our research utilizes the input-output conformance (ioco) [25] and represents specifications as Symbolic Transition Systems (STSs). The STS is developed from the Labeled Transition System (LTS). We use the definitions of LTS and STS proposed by Tretmans [25] and Frantzen et al. [12], respectively.

The LTS consists of *states* (denoted q) interconnected by *transitions*. Transitions carry a *label* specifying the action required to fire that transition. Input actions of a system are denoted with a $?$, while output actions are denoted with a $!$. Each LTS has an initial state, q_0 . Fig. 1a shows an example LTS of the coffee machine from subsection 2.1.

The STS extends the LTS with symbolic data variables [12]. The data is included by means of state variables, which are global to the STS. States in the STS are denoted with l . Transitions not only carry a label but can also include a guard (or *gate*) and an update statement. A guard resembles a boolean constraint: it must be satisfied in order for the transition to be able to fire. Guards can constrain state variables, as well as interaction variables that are local to the transition itself. An update statement assigns values to state variables. Fig. 1b shows the coffee machine example translated to an STS.

An STS captures the behavior of a SUT and can be used to test its correctness using the ioco relation. A SUT i is in ioco with a model m if the following two conditions hold [12]: (1) if i produces an output x after a trace σ , then m can also produce x after σ ; (2) if i cannot produce any output after a trace σ , then neither can m . Here, an output x may represent either an actual response or the absence of outputs, known as quiescence. If the model expects quiescence while the SUT produces an output, the ioco relation is violated. Conformance is checked by executing the same input trace on both the model and SUT, and comparing the observed outputs. These traces are obtained by traversing the model state space. The chosen traversal strategy determines which input transitions are explored. *We design the traversal strategy specifically to satisfy a given observation objective.*

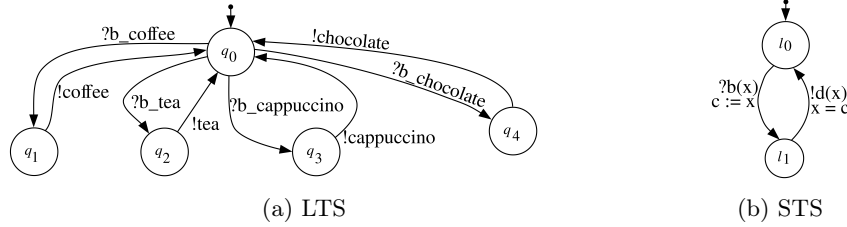


Fig. 1: Coffee machine models

2.3 The state space problem

MBT can be viewed as a graph-exploration problem in which test cases are generated by traversing a behavioral model. When test purposes are used, test generation becomes a reachability problem: finding traces that satisfy given observation objectives. The efficiency of this process depends on the model's structure. This makes exhibition-driven MBT prone to the state space explosion problem. State-space explosion is a major challenge in testing and verification [9]. Key factors contributing to state space are: (i) the number of variables and their domain sizes, which directly determine the number of possible state valuations [9,21]; (ii) the number of interacting components or processes, where the global state

space is the product of local state spaces [11]; (iii) concurrency and the resulting combinatorial number of possible interleavings of actions [30]; and (iv) nondeterminism, which introduces branching in the state space and increases the number of execution traces that must be explored [17].

These factors apply to many real-world systems. For example, microservice-based applications combine distributed state, asynchronous communication, concurrency, and nondeterminism, yielding large state spaces that are difficult to explore exhaustively. MBT applications tackle the state space explosion problem in two ways: (i) by using symbolic data and (ii) by testing on-the-fly.

MBT applications typically address this challenge in two ways: symbolic modeling and online testing. Symbolic models such as STSs represent data symbolically rather than encoding all data valuations explicitly, thereby reducing the structural state space [8]. Compared to LTSs, STSs separate control flow from data and therefore scale better to data-intensive systems. The difference in structural state space can already be observed in Fig. 1.

Another way to address the state space problem in MBT is through online testing, where test cases are generated and executed incrementally [16,28,20]. Unlike offline approaches that construct complete test cases beforehand, online testing explores the model on-the-fly and adapts to observed behavior. This reduces the need for exhaustive state-space exploration and is particularly effective in the presence of large data domains and nondeterministic behavior [16,28].

3 Related work

Over the years, many different model-based testing methods that focus on conformance have been proposed. These are all MBT strategies that use a test purpose as an additional input to steer test case generation. Oftentimes, this test purpose is modeled as its own, smaller, formal model [16,5,4,10,2,1,20,27,8].

A common strategy is to construct the synchronous product of the specification and the test purpose, and subsequently perform reachability analysis on the resulting model [10,2,20,14,8]. These approaches generate tests offline and therefore require exploration of the combined state space before execution. As a result, they remain vulnerable to state-space explosion. For example, Andrade et al. [2] model both the specification and test purpose as timed input-output STSs, compute their synchronous product, and perform symbolic reachability analysis to derive test cases. Their strategy only generates the traces but cannot execute them; this has to be done by a separate tool. Here lies a disadvantage when testing non-deterministic systems, as the test case generated by the offline tool could very well not be executable on the system.

Several online methods exist that explore the state space of the model on-the-fly. The methods most closely related to our work are TGV [16] and TorXakis [27]. TGV performs online test generation but relies on LTS models, requiring data to be encoded explicitly in the state space. TorXakis combines online testing with symbolic STS models and supports test purposes. However, while TorXakis as a tool itself is well documented, the test purpose feature is under-

exposed. From all of the published work, it is not clear *how* test purposes are used to steer the test case generation [6]. Our work adopts online testing and symbolic models, and explicitly investigates and evaluates a test purpose-driven strategy as compared to other MBT strategies.

An alternative line of work embeds the test purpose directly into the specification model rather than representing it separately [29,13], such as the Test Reactive Planner (TRP) [28]. In this MBT method, the specification model is described using an output observable nondeterministic extended finite state machine (EFSM). The test purpose is defined as a set of traps, where a trap is a boolean variable on a transition of the EFSM, that is false unless that transition has fired. Test generation then aims to cover all traps at least once. Although computationally attractive, such trap-based approaches are less expressive than model-based test purposes because they capture coverage objectives and not temporal ordering constraints.

Some approaches specifically target concurrent systems [15,20,19]. Concurrent systems suffer even more from state space explosion due to the parallel nature of their actions. Jard [15] addresses this problem by using unfolding techniques to avoid enumerating all action interleavings when generating distributed tests. Although our proposed method could be applied to concurrent systems, it does not specifically optimize for concurrency.

Despite substantial academic interest in test purpose-based MBT, industry MBT applications predominantly employ structural coverage-based or random strategies [18]. Random strategies are a random walk over the model: they simply pick any of the currently available transitions, preferring none over the other. Structural coverage-based strategies aim to optimize the coverage of a structural property of the model. Well-known examples are state and transition coverage. These strategies will prefer to take routes through the model that explore different states and transitions, preferring those states/transitions that it has not covered before. Examples of industry tools include the Axini Modeling Platform, which provides `Random` and `GlobalTransitionCoverage` strategies [22], and Microsoft’s offline MBT tool, which generates traces either completely randomly or towards a specific model state [23].

An example of an industry-tool that does include exhibition-driven MBT is GraphWalker. GraphWalker offers a requirement-based feature ¹ in which model elements are tagged with requirements, and test generation aims to cover all tagged elements. Similarly to trap-based approaches such as TRP, this mechanism expresses coverage goals rather than ordered behavioral objectives.

4 Method: CONEXA

We propose **Conexa** (Conformance-Exhibition Algorithm), an online MBT strategy that uses a specification model and a test purpose model to guide test generation. Both are modeled as STSs, to reduce the state space addressed by the

¹ <https://github.com/GraphWalker/graphwalker-project/wiki/Requirements>

graph traversal algorithm. The test purpose model contains one final accepting state. Once that state is reached, the observation objective of that test purpose has been exhibited.

Test purposes. We use test purposes to serve observation objectives and exhibition, and to map system requirements to test generation. To this end, the behavioral model of the test purpose should be as expressive as the behavioral model of the SUT. For **Conexa**, we model the specification and the test purpose as STSs. In addition to regular STS constructs, a test purpose contains a single accepting state (denoted as a double circle) and three wildcard transitions: $?any$, $!any$, and $!?any$. Wildcard transitions match any (input/output) transition of the model. The accepting state marks successful exhibition of the observation objective. Any trace that reaches the accepting state, therefore, hits the test purpose’s observation objective.

Conexa takes a single test purpose as input and executes it along with the specification model during testing. Consequently, it maintains the set of active states for both models. Following ioco semantics, these are the states reachable after executing the trace generated thus far. Due to nondeterminism and wildcard matching, multiple states may be active simultaneously. Figure 4b shows an example test purpose.

Choosing input transitions. Figure 2 summarizes **Conexa**. The main logic of **Conexa** covers the Testing Phase, i.e., steps 2-5 in Fig. 2. During this phase, **Conexa** keeps track of the active state(s) of the specification model and the test purpose model. The Testing Phase is all about selecting the “right” transition from the currently available specification model transitions.

Intuitively, a transition is “right” when it advances the test purpose model closer to its accepting state. For a strategy that focuses on optimizing transition coverage, the “right” transition would be one that has not been covered during this test run. A strategy that uses a random walk, the “right” transition is any random transition. **Conexa** prefers transitions that progress exhibition.

A transition is selected when: (i) its label matches an enabled non-wildcard transition in the test purpose, and (ii) taking it yields at least one previously unseen (model transition, test purpose state) pair. The first transition satisfying both criteria is chosen immediately. If no enabled transition satisfies them, **Conexa** performs a bounded look-ahead until a suitable transition is found.

Once the accepting state is reached, the test purpose is reset to its initial state while the specification model remains unchanged. Testing then continues to discover alternative traces that exhibit the same observation objective. These traces can be beneficial when hunting for bugs, as not all bugs are bound to a specific location in a model; some are bound to a sequence of steps [7].

4.1 Heuristics and improvements

To improve the generated traces, both in terms of exhibition and in terms of variability among them, we add some heuristics to the main logical flow of **Conexa**. We also note that structuring the test purpose in a specific way can improve the performance of **Conexa**.

Conexa — How the next transition is selected

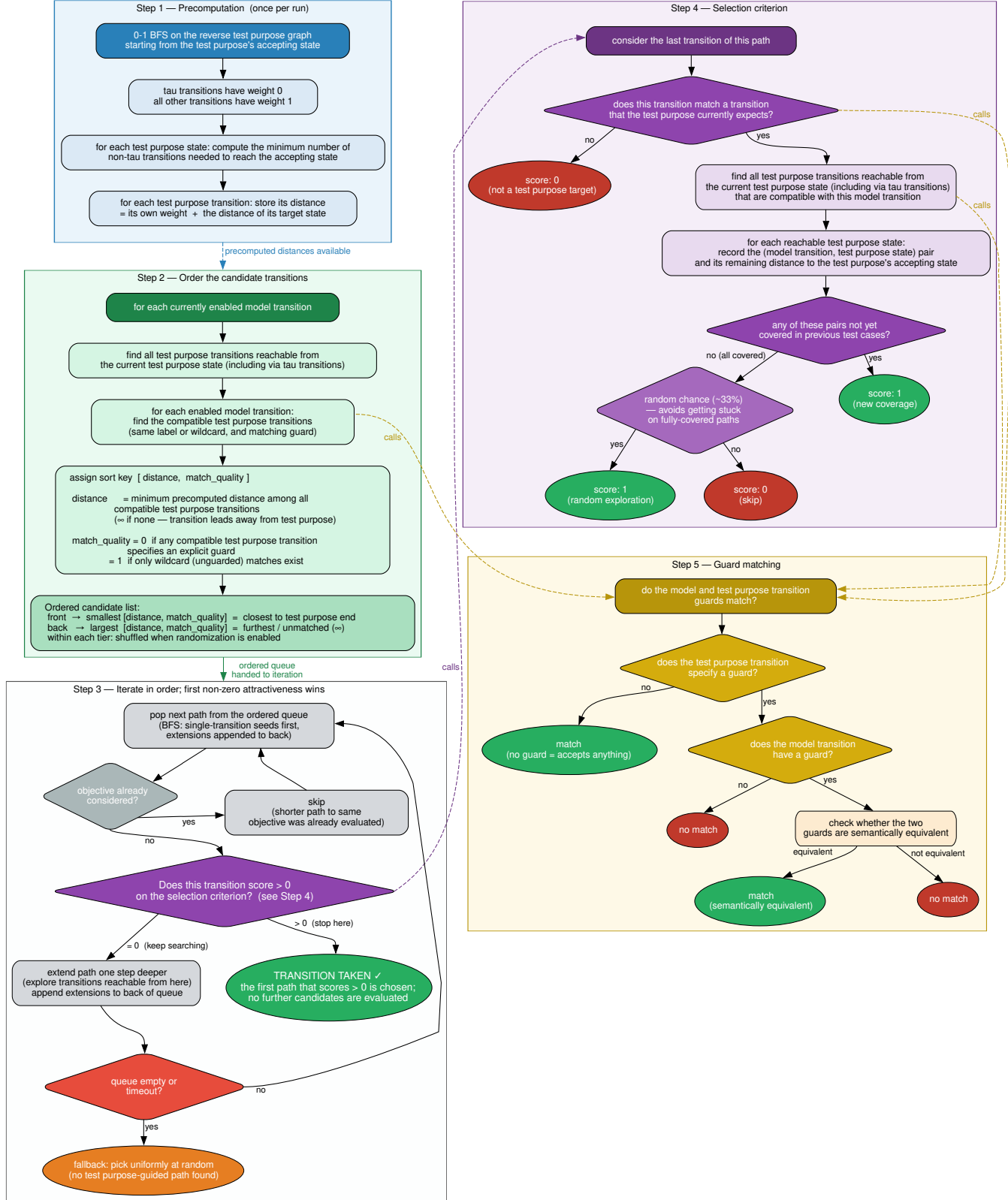


Fig. 2: Conexa - Selecting an input transition

Transition order manipulation. In Step 2 of Fig. 2, **Conexa** manipulates the order in which it considers the available transitions based on two factors: the minimal distance the test purpose still needs to travel, and whether the guard of the model transition matches the test purpose’s.

Test purposes can describe different routes to the same end-goal, or they can include optional behavior that is allowed, but not strictly necessary for exhibition. This means that you can construct accepting traces of different sizes. For efficiency considerations, **Conexa** includes a Preparation Phase (Step 1 in Fig. 2), where the test purpose distance is calculated. This step is done only once before the test case generation starts. In the Preparation Phase, **Conexa** computes the shortest distance from every test purpose state to the accepting state using a 0–1 BFS traversal. This preprocessing is performed only on the test purpose model; the specification model remains explored entirely on-the-fly. These distances are used to guide the search later on: a transition that advances the test purpose closer to its acceptance state is preferred over one that does not. After this preparation step, the Testing Phase starts.

In the Testing Phase, **Conexa** assesses the match quality of enabled model transitions. Match quality is determined by two factors: matching labels and matching guards. High quality transitions match on both. Model transitions have low match quality when: (i) their label does not match the label of any of the test purpose’s available non-wildcard transitions, or (ii) their label does match but the guard does not, or (iii) their label does match but the test purpose’s transitions have no guard at all. The last criterion promotes guarded transitions over unguarded ones, which helps with effective data solving.

During testing, **Conexa** combines the distance and match quality metrics to manipulate the order in which it considers the available model transitions. First, the transitions are ranked by the distance of their matching test purpose transitions. Then, within the same distance tier, transitions that match explicit guards are prioritized over wildcard matches or unguarded transitions. This ordering increases the likelihood of progressing efficiently towards exhibition.

Visited Pairs and Coinflip. To encourage trace diversity, **Conexa** records every observed (model transition, test purpose state) pair. Whenever a model transition is taken, **Conexa** advances the test purpose using the same label, and looks at which test purpose states are now active. For each active state, **Conexa** stores a pair of (the model transition that was just taken, the test purpose state that is now active). When considering whether to take a transition, those transitions leading to previously unseen pairs are preferred over already observed ones. This promotes the exploration of different traces.

However, revisiting an existing pair remains possible with a probability of 33% (see Step 4 in Fig 2). This coin-flip mechanism prevents the search from becoming too restrictive in situations where only a limited number of transitions can advance the test purpose. Together, the visited-pair heuristic and coin flip balance exploration of alternative traces with progress towards exhibition.

Better test purposes. Because the specification model is explored on-the-fly, **Conexa** has limited knowledge about which model transitions best advance an

observation objective. Additional guidance can therefore be encoded in the test purpose itself. Figure 3 shows three test purposes for the objective “the coffee machine can dispense cappuccino”. Test purpose (a) specifies only the desired output, test purpose (b) adds the required input action, and test purpose (c) additionally constrains the relevant input data through a guard.

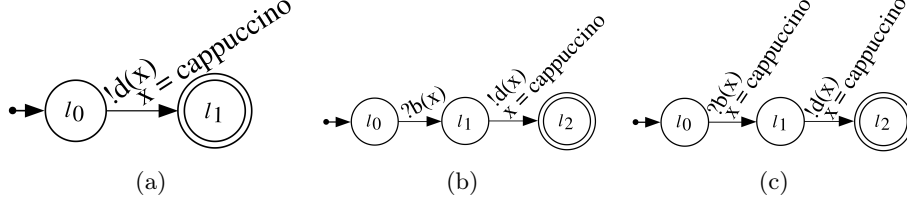


Fig. 3: Different ways to demote the same observation objective

Under ioco, outputs cannot be controlled directly; they can only be observed. Consequently, test purpose (a) provides insufficient guidance. Test purpose (b) identifies the relevant input, but leaves data selection entirely to the solver. Test purpose (c) explicitly constrains the data and therefore provides the information required to steer the search towards the desired behavior. In general, test purposes become more effective when they describe not only the target behavior but also the conditions needed to reach it.

4.2 ATM example

We now use Figure 4 to illustrate **Conexa** on an ATM model and the corresponding test purpose. The observation objective is to verify that the reported balance changes after a withdrawal. The initial **!any** transition in the test purpose ensures that any leading sequence can match it, which in turn means that there will always be at least one active state.

Conexa first computes the test purpose distances, shown in Fig. 4b. It then repeatedly selects enabled model transitions that best match the expected non-wildcard test purpose transitions. For example, it first chooses **?request_balance**, followed by **?withdraw(x)**. Later, when both **?continue** and **?bill_preference** are available, **Conexa** prefers **?continue** because its distance is smaller. Upon reaching the accepting state, the test purpose is reset while the specification model remains unchanged. The visited-pair heuristic then discourages revisiting the same transition sequence. Consequently, when the same decision point is encountered, **Conexa** may select **?bill_preference** instead of **?continue**, producing a different trace to the same observation objective.

5 Experimental setup

We evaluate **Conexa** through a controlled experiment. The goal is twofold: (i) to determine whether **Conexa** can generate traces that exhibit requirement-specific observation objectives, and (ii) to compare these traces against those generated

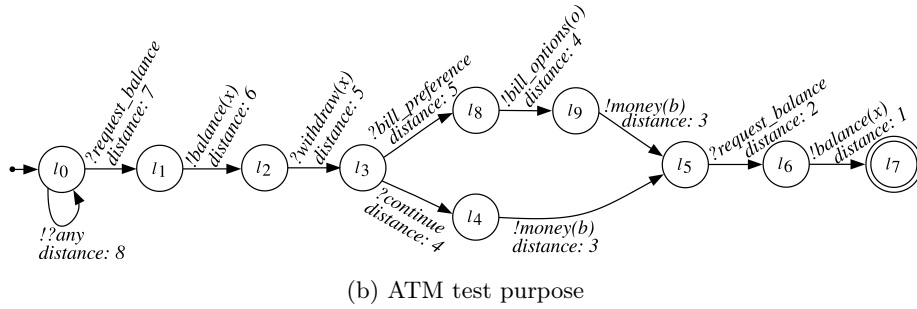
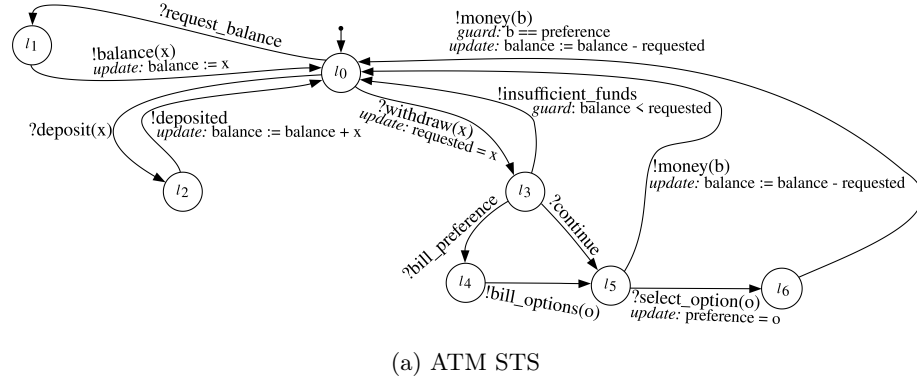


Fig. 4: Conexa ATM example

by industry-standard MBT strategies, namely random and structural coverage-based testing. Evaluation is performed on the SmartDoor, a benchmark system used to teach model-based testing to industry practitioners. We implement **Conexa** as a test-generation strategy in a commercial MBT tool. A test run consists of multiple test cases, each with a user-defined maximum number of steps, where each input and output counts as one step. Each test run uses a single test purpose. Figure 5 shows the experimental setup.

Test generation is performed online, and the chosen transitions are executed immediately through an adapter connecting the MBT tool and the SUT. Data constraints on these transitions are made concrete through a solver (Z3). Because test generation occurs during execution, the strategy operates under a time constraint. A deadline is set at the start of **Conexa**'s transition selection process. If no transition has been selected when this deadline expires, a random transition is taken. See Step 3 in Fig. 2.

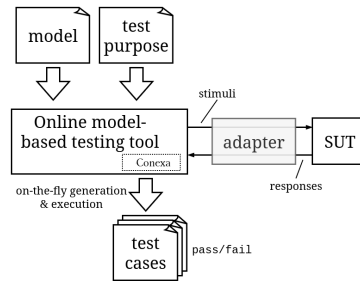


Fig. 5: Experimental setup

5.1 SmartDoor

SmartDoor is an IoT door system used for MBT training. It can be opened, closed, locked, and unlocked via an app. Locking and unlocking require a passcode, and three consecutive failed unlock attempts cause the door to shut off. Communication between the app and the door takes place through message exchanges defined by the specification. An STS model of SmartDoor is available in the artifact accompanying this study². The SmartDoor consists of a specification document that describes the requirements, a corresponding SUT, and SUT mutants that contain bugs. For our experiment, we only use the faultless SUT.

The SmartDoor specification contains a collection of behavioral requirements that we translate to test purposes. Table 2 lists five representative requirements used in this paper. Requirement **SECL0C-07** is modeled using two test purposes: one describing the complete behavioral sequence leading to shutdown, and one describing only the final **shut_off** observation. These examples illustrate the effect of test purpose design on **Conexa**’s performance. Although only five requirements are discussed in detail, we conduct the experiment on all behavioral requirements in the SmartDoor specification. The complete dataset, models, and test purposes are included in the study artifact.

Table 2: SmartDoor requirements

BEHAVR-02	When the door is opened, it may be closed.
BEHAVR-04	When the door is closed and locked, it may be unlocked.
SECL0C-03	When the lock command is used with a valid passcode, the door can only be unlocked with an unlock command containing that exact passcode. Other passcodes are considered incorrect.
SECL0C-05	When the unlock command is used with an incorrect passcode, the command must be refused and then door must return the <code>incorrect_passcode</code> response.
SECL0C-07	When the unlock command is used three times with an incorrect passcode, the door must shut off and not respond to any commands until it is restarted physically.

6 Results

We first evaluate the exhibition performance, i.e. the ability of a strategy to generate traces that satisfy a test purpose. We then compare the length of the traces generated by **ConEx**’ against the shortest possible traces obtained by offline analysis.

6.1 Exhibition

We evaluate three test-generation strategies: random, transition coverage, and **Conexa**. For each strategy, 10 test runs are executed. Each run consists of three test cases with a maximum length of 100 steps. For **Conexa**, the corresponding

² GitHub link removed for double-blind review.

test purpose is provided as an additional input. We translate the five behavioral requirements in Table 2 to six test purposes. Requirement SECL0C-07 is modeled twice: once as a detailed behavioral test purpose, and once as a simplified objective (`‘the door can shut off’`) that only specifies the desired final observation. For each strategy and test purpose, Fig. 6 reports:

Runs with a hit: Number of test runs (out of 10) in which the observation objective was exhibited at least once

Hits per hitting run: Average number of successful exhibitions among runs that achieved at least one hit.

Steps per hit: Average number of steps until exhibition.

6.2 Trace length: comparing offline approaches and Conexa

Offline exhibition-driven MBT methods compute traces after exploring the relevant state space and can therefore identify shortest paths to exhibition. In contrast, online strategies like **Conexa**, explore the state space of the model on-the-fly. This raises the question: *How close are the traces found by Conexa to the shortest possible traces?*

Table 3 compares the length of traces generated by **Conexa** with the optimal trace length obtained by offline analysis. For all test purposes and across all test runs, the first trace found by **Conexa** is also the shortest trace found. The only exception is the test purpose `‘the door can shut off’`, where **Conexa** found traces of varying length (14-53 steps), with an average length of 31 steps.

Table 3: Number of steps per hitting trace

	Conexa		Offline
	First trace found	Shortest trace found	Optimal trace
BEHAVR-02	4	4	4
BEHAVR-04	4	4	4
SECL0C-03	4	4	4
SECL0C-05	4	4	4
SECL0C-07	9	9	9
The door can shut off	31	14	9

7 Discussion

To judge the performance of **Conexa**, we look at how well it can make a system exhibit. We see that for all five fully specified test purposes, **Conexa** has at least as many ‘runs with a hit’ as the baseline strategies. The hits per hitting run are higher for **Conexa** as well. Additionally, the total steps per hit are lower. This means that **Conexa** can make a SUT exhibit an observation objective in fewer steps, as well as more frequently, than traditional model-based testing strategies.

The effectiveness of **Conexa** however, depends strongly on the quality of the test purpose. This is illustrated by the difference in performance between the test purposes SECL0C-07 and `‘the door can shut off’`. The former is modeled according to the guidelines proposed in this paper, whereas the latter specifies

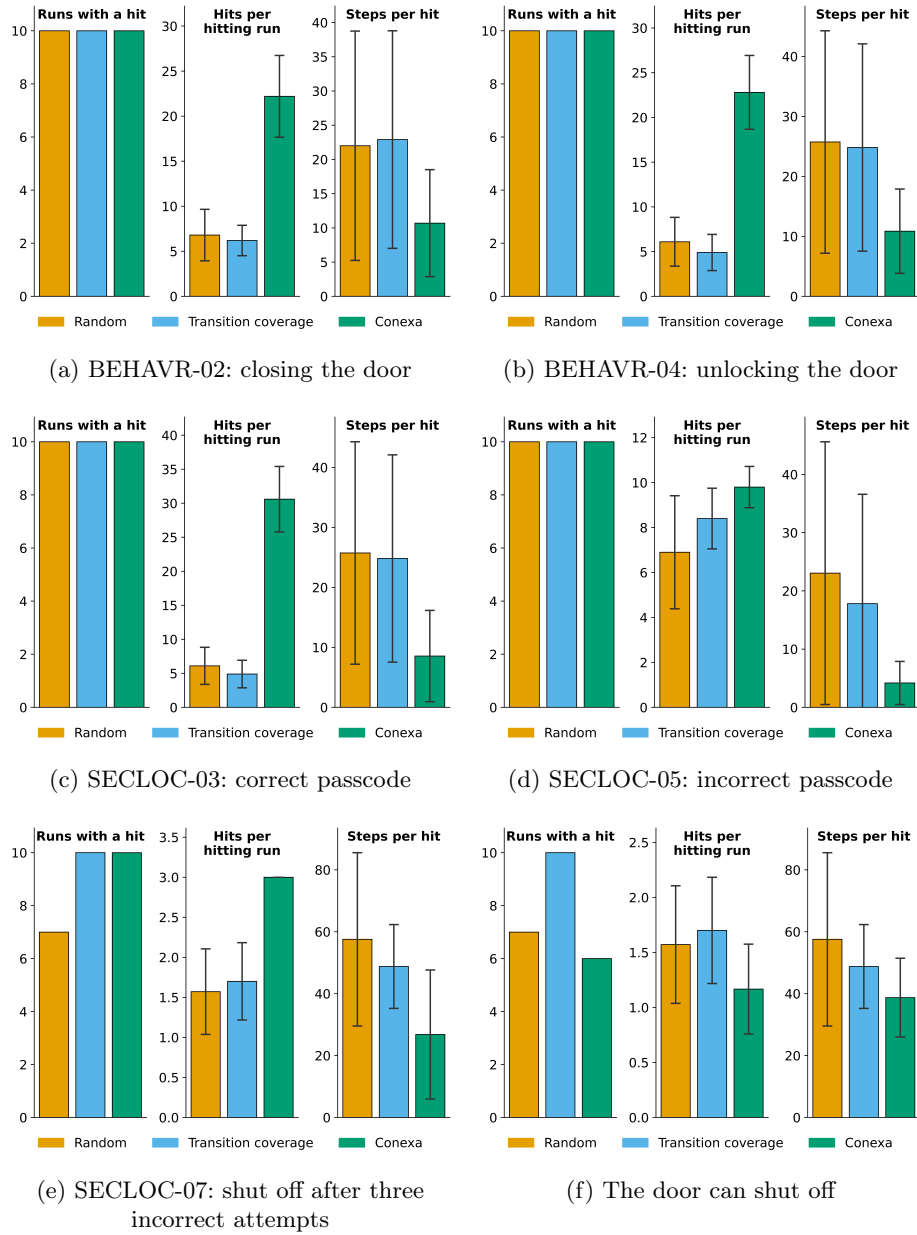


Fig. 6: Hits per strategy

only the final observation. For SECLOC-07, Conexa clearly outperforms the baseline strategies. For ‘the door can shut off’, its performance is comparable to random testing and inferior to transition coverage. These results indicate that the success of exhibition-driven testing depends not only on the test generation strategy, but also on the expressiveness of the test purpose used to guide it.

A key motivation for designing **Conexa** is mitigating state space explosion through online test generation. Despite operating on-the-fly, **Conexa** produced optimal traces for all well-modeled test purposes. Only the under-specified test purpose ‘**the door can shut off**’ yields suboptimal (i.e. longer) traces, indicating that online test generation can approach offline performance when sufficient guidance is available in the test purpose.

7.1 Challenges in online exhibition-driven testing

The main challenge in our online exhibition-driven MBT strategy is the absence of a natural distance metric over the specification model. Because **Conexa** does not precompute synchronized reachability information, it cannot estimate how far a model state is from satisfying a test purpose. Instead, transitions are evaluated locally based on whether they advance the test purpose with one step.

This limitation becomes particularly apparent when progress depends on non-local data dependencies or longer action sequences. When test purpose transitions are guarded by data constraints that are non-local, dependencies between the chosen action and overall goal satisfaction (i.e. test purpose acceptance) exist. In the SmartDoor example, the **!shut_off** response can only be observed after three consecutive incorrect unlock attempts. Whether the current state of the model can satisfy that guard depends on actions that were taken previously. In other words, the data dependency is non-local and therefore not anticipated and prepared for by the online strategy.

Similarly, a model transition label may match the test purpose’s label, while leading to behavior that does not contribute to eventual exhibition. This case is also reflected by the SmartDoor: imagine a scenario where a test purpose wants to see **?close** followed by **!closed**, then **Conexa** greedily searches for a **?close** transition. However, when the door is already closed, this input leads to **!invalid_command**, which does not advance the test purpose any further. Consequently, advancing the test purpose locally does not always correspond to advancing towards global goal completion.

To compensate for the lack of a natural distance metric, **Conexa** relies on expressive test purposes and a set of heuristics. The results show that these mechanisms are effective, but cannot provide the same guarantees as offline reachability analysis. In practice, richer test purposes lead to faster and more reliable exhibition. This introduces a trade-off: the more information encoded in the test purpose, the more effective the search becomes, but the closer the test purpose moves towards describing a predefined test scenario (e.g. a BDD), rather than a testing objective. To preserve exploratory MBT behavior, **Conexa** continues to search for alternative traces after exhibition has been achieved once.

Another limitation is that **Conexa** targets one test purpose at a time. Achieving broad requirement coverage therefore requires multiple test runs. This contrasts with structural coverage strategies, which attempt to cover a lot of different behavior within a single run. This could be inefficient. However it can be offset by the fact that you need fewer steps per test case, and therefore possibly also

fewer test cases. The two approaches are complementary: coverage-based testing provides broad exploration, whereas exhibition-driven testing targets specific requirements with high precision.

This distinction aligns with common MBT usage scenarios. During active development, broad exploration may be preferable for discovering unexpected behavior or bugs. During regression testing and maintenance phase, where requirements must be verified repeatedly, exhibition-driven testing can provide a more focused and efficient testing strategy.

8 Conclusion

This paper introduced **Conexa**, an online model-based testing strategy that combines conformance testing with exhibition-driven test generation. **Conexa** uses test purposes to guide model exploration towards observation objectives while exploring the specification model on-the-fly. To improve performance, we introduce a set of heuristics and showed that the formulation of test purposes has a significant impact on the effectiveness of the strategy.

The experimental results demonstrate that **Conexa** can exhibit observation objectives more frequently and in fewer steps than random and transition coverage-based testing. Moreover, despite operating online, **Conexa** produces optimal traces for all sufficiently informative test purposes. These results suggest that online exhibition-driven testing can mitigate state-space exploration while remaining effective at reaching targeted behaviors. However, the effectiveness of **Conexa** depends on the quality of the test purpose: objectives that provide little guidance are more difficult to exhibit efficiently.

8.1 Future Work

Although **Conexa** is evaluated against conventional MBT strategies, we have not yet compared it directly with other exhibition-driven approaches. Future work should therefore include comparative evaluations against tools such as TorXakis, TGV, or GraphWalker.

There are several opportunities to improve **Conexa** itself. The specification model and the test purpose could share data during test generation, additional heuristics for transition selection could be developed, and the strategy could be extended to steer towards multiple test purposes within a single test run.

Finally, more research is needed to better understand the trade-offs between online and offline MBT exhibition-driven methods. Although we have a strong intuition that the online approach mitigates the state space explosion, this is not quantified in the present study. Similarly, the circumstances under which exhibition-driven testing should be preferred to structural coverage-based testing are unclear. Comparative case studies could investigate these trade-offs in both bug-finding and regression-testing scenarios.

References

1. Andrade, W.L., Machado, P.D.L.: Testing interruptions in reactive systems. *Formal Aspects of Computing* **24**(3), 331–353 (2011). <https://doi.org/10.1007/s00165-011-0197-0>
2. Andrade, W.L., Machado, P.D.L., Jéron, T., Marchand, H.: Abstracting time and data for conformance testing of real-time systems. In: *Proceedings of ICSTW 2011*. pp. 9–17. IEEE (2011). <https://doi.org/10.1109/ICSTW.2011.82>
3. Apfelbaum, L., Doyle, J.: Model based testing. In: *Software quality week conference*. pp. 296–300 (1997)
4. Bannour, B., Lapitre, A., Le Gall, P., Nguyen, T.: Symbolic path-guided test cases for models with data and time. *arXiv* (2023). <https://doi.org/10.48550/arXiv.2309.06840>
5. Bonifácio, A., Moura, A.V.: Testing asynchronous reactive systems: Beyond the ioco framework. *CLEI Electronic Journal* **24**(2) (2021). <https://doi.org/10.19153/cleiej.24.2.13>
6. van den Bos, P., Tretmans, J.: Coverage-based testing with symbolic transition systems. In: *International Conference on Tests and Proofs*. pp. 64–82. Springer (2019)
7. van den Broek, S., Oprescu, A., Ruys, T.C.: Bug localisation in model-based testing using model-centric shrinking. In: *Proceedings of the 2nd ACM International Workshop on Future Debugging Techniques*. p. 9–15. DEBT 2024, Association for Computing Machinery, New York, NY, USA (2024). <https://doi.org/10.1145/3678720.3685315>, <https://doi.org/10.1145/3678720.3685315>
8. Clarke, D., Jéron, T., Rusu, V., Zinovieva, E.: Stg: A symbolic test generation tool. In: *Lecture Notes in Computer Science*. pp. 470–475. Springer (2002). https://doi.org/10.1007/3-540-46002-0_34
9. Clarke, E.M., Klieber, W., Nováček, M., Zuliani, P.: Model checking and the state explosion problem. In: *LASER Summer School on Software Engineering*, pp. 1–30. Springer (2011)
10. Constant, C., Jéron, T., Marchand, H., Rusu, V.: Integrating formal verification and conformance testing for reactive systems. *IEEE Transactions on Software Engineering* **33**(8), 558–574 (2007). <https://doi.org/10.1109/TSE.2007.70707>
11. Demri, S., Laroussinie, F., Schnoebelen, P.: A parametric analysis of the state-explosion problem in model checking. *Journal of Computer and System Sciences* **72**(4), 547–575 (2006)
12. Frantzen, L., Tretmans, J., Willemse, T.A.: Test generation based on symbolic specifications. In: *International Workshop on Formal Approaches to Software Testing*. pp. 1–15. Springer (2004)
13. Halling, E., Vain, J., Boyarchuk, A., Illiashenko, O.: Test scenario specification language for model-based testing. *International Journal of Computing* (2019)
14. Henry, L., Jéron, T., Markey, N.: Control strategies for off-line testing of timed systems. *Formal Methods in System Design* **60**(2), 147–194 (2022). <https://doi.org/10.1007/s10703-022-00403-w>
15. Jard, C.: Synthesis of distributed testers from true-concurrency models of reactive systems. *Information and Software Technology* **45**(12), 805–814 (2003). [https://doi.org/10.1016/s0950-5849\(03\)00061-2](https://doi.org/10.1016/s0950-5849(03)00061-2)
16. Jard, C., Jéron, T.: Tgv: Theory, principles and algorithms. *International Journal on Software Tools for Technology Transfer* **7**(4), 297–315 (2004). <https://doi.org/10.1007/s10009-004-0153-x>

17. Laroussinie, F., Sproston, J.: State explosion in almost-sure probabilistic reachability. *Information Processing Letters* **102**(6), 236–241 (2007)
18. Li, W., Le Gall, F., Spaseski, N.: A survey on model-based testing tools for test case generation. In: *International Conference on Tools and Methods for Program Analysis*. pp. 77–89. Springer (2017)
19. Liu, J., Ye, X., Zhou, J.: Test purpose oriented i/o conformance test selection with colored petri nets. *Journal of Applied Mathematics* pp. 1–10 (2014). <https://doi.org/10.1155/2014/645235>
20. Marsso, L.: On Model-Based Testing of GALS Systems. Ph.D. thesis, Université Grenoble Alpes (2019), PhD thesis, NNT: 2019GREAM078
21. Pelánek, R.: Properties of state spaces and their applications. *International Journal on Software Tools for Technology Transfer* **10**(5), 443–454 (2008)
22. Ruys, T.C., van der Bijl, M.: Effective model-based testing. In: *Principles of verification: Cycling the probabilistic landscape: Essays dedicated to Joost-Pieter Katoen on the occasion of his 60th birthday, part III*, pp. 234–251. Springer (2024)
23. Stobie, K.: Model based testing in practice at microsoft. *Electronic Notes in Theoretical Computer Science* **111**, 5–12 (2005)
24. Tretmans, G.: A formal approach to conformance testing. Phd thesis - research ut, graduation ut, University of Twente, Netherlands (1992). <https://doi.org/10.3990/1.9789090056432>
25. Tretmans, J.: Model based testing with labelled transition systems. In: *Formal methods and testing: An outcome of the FORTEST network, revised selected papers*, pp. 1–38. Springer (2008)
26. Tretmans, J.: Model-based testing and some steps towards test-based modelling. In: *International School on Formal Methods for the Design of Computer, Communication and Software Systems*, pp. 297–326. Springer (2011)
27. Tretmans, J.: On the existence of practical testers. In: *Modeled, Tested, Trusted: Essays Dedicated to Ed Brinksma on the Occasion of His 60th Birthday*, pp. 87–106. Springer (2017)
28. Vain, J., Kääramees, M., Markvardt, M.: Online testing of nondeterministic systems with the reactive planning tester. In: *Advances in Computer and Electrical Engineering*, pp. 113–150. IGI Global (2011). <https://doi.org/10.4018/978-1-60960-747-0.ch007>
29. Vain, J., Raiend, K., Kull, A., Ernits, J.P.: Synthesis of test purpose directed reactive planning tester for nondeterministic systems. In: *Proceedings of ASE 2007*. ACM (2007). <https://doi.org/10.1145/1321631.1321685>
30. Valmari, A.: The state explosion problem, pp. 429–528. Springer Berlin Heidelberg, Berlin, Heidelberg (1998). https://doi.org/10.1007/3-540-65306-6_21, https://doi.org/10.1007/3-540-65306-6_21
31. de Vries, R.G.: Towards formal test purposes. In: *1st International Workshop on Formal Approaches to Testing of Software, FATES 2001*. pp. 61–76. BRICS, University of Aarhus (2001)